SEN4100 – Software Project Management

Week 5 Lecture Notes: Cost & Effort Estimation and Introduction to Scheduling

- **Course Learning Outcomes (CLOs) Addressed:**
  - Apply scheduling, budgeting, and resource-allocation techniques.
  - Evaluate project performance using industry-standard metrics.
- **Primary Textbooks:** Pressman & Maxim (*Software Engineering: A Practitioner's Approach*), Hughes & Cotterell (*Software Project Management*).

1. The Context of Estimation in Software Engineering

In the previous phase, we utilized the **Work Breakdown Structure (WBS)** to decompose the project scope into assignable Work Packages.

Decomposition answered the question: *"What are the specific tasks required?"* Estimation answers the question: *"How big is this system? How much effort will it take? How much will it cost?"*

Estimation is notoriously difficult in software engineering because software is an intangible intellectual product, not a physical manufacturing process. The "Cone of Uncertainty" dictates that at the beginning of a project; estimates can vary by as much as 400%. Only by applying structured algorithmic and empirical models can a Software Project Manager (SPM) narrow this variance.

2. Software Sizing Techniques

Before estimating effort (time) and cost (money), we must calculate the fundamental **size** of the software. We cannot predict how long it takes to paint a building until we know its square footage.

2.1 Lines of Code (LOC) Estimation

- **Concept:** Historically the oldest metric, LOC estimates size by predicting how many lines of source code will be required to build the system (often measured in KLOC - Thousands of Lines of Code).
- **Advantages:** It is highly intuitive for developers and easily automated. Once the software is written, LOC can be counted instantly.
- **The "Productivity Paradox" (Fatal Flaw):**
  - LOC is heavily dependent on the programming language. Implementing a sorting algorithm might take 100 lines in Assembly, 20 lines in C++, and 3 lines in Python.
  - LOC punishes efficiency. A senior software engineer who spends three days refactoring 2,000 lines of brittle code into 300 lines of elegant, maintainable code appears to have

"negative productivity" under LOC metrics.
- Clients pay for business functionality, not for the volume of text typed by developers.

2.2 Function Point (FP) Analysis

Developed by Allan Albrecht at IBM, FP measures software size based on the **functionality requested by and provided to the end-user**, making it entirely independent of the programming language.

The 5 Information Domains: To calculate Function Points, the system is analyzed across five external and internal domains:

1. **External Inputs (EI):** Screens, forms, or data streams where the user inputs data into the system (e.g., a "Create New User" form).
2. **External Outputs (EO):** Screens, reports, or graphs where the system presents calculated data to the user (e.g., a "Monthly Sales Dashboard").
3. **External Inquiries (EQ):** Simple read-only data retrievals where no data is altered (e.g., "Search for User by ID").
4. **Internal Logical Files (ILF):** Data tables or databases maintained entirely within the system's boundary (e.g., a localized PostgreSQL 'Users' table).
5. **External Interface Files (EIF):** Databases or APIs maintained by other systems but read by your system (e.g., reading a currency exchange rate from a central bank API).

Calculation Mechanics:

1. Each of the 5 domains is counted.
2. The counts are weighted based on complexity (Simple, Average, Complex).
3. The sum yields the **Unadjusted Function Point (UFP)** count.
4. The UFP is then modified by 14 **Value Adjustment Factors (VAF)** (e.g., Does the system require high transaction rates? Is it heavily distributed? Are end-user interfaces exceptionally complex?).

Conclusion: Function points are superior to LOC for client contracts because they measure the actual business value being delivered.

3. Algorithmic Cost Modeling: COCOMO

Once the size of the software is determined (either via KLOC or FP converted to KLOC), the SPM must convert that size into **Effort** and **Duration**.

The most widely taught algorithmic model is the **COnstructive COst MOdel (COCOMO)**, developed by Dr. Barry Boehm. It is an empirical model derived by statistically analyzing thousands of past software projects.

3.1 The Three COCOMO Project Modes

COCOMO recognizes that building a simple HR tool is fundamentally different from building an avionics flight-control system. It categorizes projects into three modes, altering the mathematical constants (a, b, c, d):

1. **Organic Mode:** Relatively small software teams working in a highly familiar, in-house domain. Communication overhead is low, and requirements are flexible. (e.g., An internal

library management system).

2. **Semi-Detached Mode:** Medium-sized projects with teams of mixed experience levels. The system contains both rigid constraints and flexible interfaces. (e.g., A university student information system like OBS).
3. **Embedded Mode:** Highly complex, tightly coupled systems operating under severe hardware, algorithmic, or regulatory constraints. Zero tolerance for failure. (e.g., Air traffic control software, FDA-regulated medical firmware).

3.2 Basic COCOMO Formulas

The basic COCOMO model calculates baseline metrics using the following structures:

Effort (E): Measured in **Person-Months (PM)**. This is the total volume of human labor required.

$$E = a_b \times (KLOC)^{b_b}$$

- *(Notice the exponent b_b. As software size grows, effort does not grow linearly; it grows exponentially due to integration and communication complexity).*

  Development Time / Duration (D): Measured in chronological **Months**. This is the calendar time required.

$$D = c_b \times (E)^{d_b}$$

- **Average Staffing Requirement:** The ideal number of full-time developers to assign to the project.

$$Staffing = \frac{E}{D}$$

- *(Note: Advanced versions like COCOMO II incorporate factors like team cohesion, personnel capability, and platform difficulty).*

4. Adaptive/Agile Estimation Techniques

Algorithmic models (COCOMO) are powerful for predictive, Waterfall-style projects where requirements are frozen upfront. However, in Agile (Scrum/Kanban) environments where requirements change rapidly, absolute hour estimations are often inaccurate. Agile teams use **relative sizing**.

4.1 Story Points

A Story Point is an abstract, relative unit of measure representing the total effort required to fully implement a User Story (Work Package). A Story Point encapsulates three dimensions:

1. **Volume:** How much physical work/code is required?
2. **Complexity:** How algorithmically or architecturally difficult is the task?
3. **Uncertainty/Risk:** How many unknown dependencies exist?

The Fibonacci Sequence: Agile teams assign Story Points using the modified Fibonacci sequence: **1, 2, 3, 5, 8, 13, 21**. *Why?* Human psychology is poor at estimating absolute values (e.g., "Will this take 42 hours or 46 hours?"), but excellent at relative comparison. If a developer looks at a baseline task rated at '3 points', they can easily judge if a new task is roughly twice as difficult (assigning it a 5 or an 8).

4.2 Planning Poker

To prevent single-point estimation failures, Agile teams use a consensus-based, gamified technique called Planning Poker during Sprint Planning:

1. The Product Owner explains the User Story to the team.
2. Developers evaluate the story and **privately** select a Fibonacci card representing their estimate.
3. All cards are revealed **simultaneously**.

The Strategic Value of Simultaneous Reveal: This mechanism explicitly eliminates **Anchoring Bias**. If estimation was done verbally, the Senior Architect might immediately say, "This is easy, it's a 2." Junior developers, fearing judgment, will silently agree. By revealing simultaneously, if a Junior plays a 13 and the Architect plays a 2, a debate is forced. The Junior might foresee an obscure database locking issue that the Architect missed.

5. Transition to Scheduling (Preview)

Estimation tells us *how many hours* the WBS packages will take. However, it does not tell us *in what order* they must be executed.

You cannot test a database before you provision the server. Therefore, tasks have **Dependencies**. Next week, we will transition to **Scheduling**, where we will take the Estimated Work Packages and map them onto a chronological timeline using:

- **Precedence Diagramming Method (AON Networks):** Defining Finish-to-Start (FS), Start-to-Start (SS) relationships.
- **Critical Path Method (CPM):** Calculating the longest path of dependent tasks through the project, which determines the absolute minimum completion time for the software.
- **Gantt Charts:** Visualizing the schedule baseline for executive reporting.

6. Alignment with Term Project: Milestone 2

Having completed the WBS (Milestone 1), your squad is now entering the preparation phase for **Milestone 2** (due Week 7 / Midterm Week).

Milestone 2 Deliverable Expectations regarding Estimation:

1. Your squad must assign an **Effort Estimate** to every Work Package at the bottom level of your WBS.
2. **Methodology Choice:**
   - If your Project Charter declared a Predictive/Waterfall model, you must use algorithmic estimation (e.g., sizing via FP and converting to hours/cost).
   - If your Project Charter declared an Agile/Scrum model, you must assign **Story Points** to your Backlog items using the Fibonacci sequence and calculate a projected Sprint

Velocity.

7. Classroom Discussion Scenarios

1. **The Brooks's Law Dilemma:**
   - You calculate via COCOMO that a project requires 100 Person-Months (E) and has a duration of 10 Months (D). The ideal staffing is 10 developers.
   - Six months into the project, you are severely behind schedule. The Executive Sponsor tells you to hire 10 more developers immediately to "catch up."
   - *Question:* According to Frederick Brooks ("The Mythical Man-Month"), why will adding 10 developers to a late software project make it later? (Consider context switching, training overhead, and communication paths: $N(N-1)/2$).
2. **Estimation Pressure:**
   - A client demands an exact cost estimate for complex AI integration before the WBS is even drafted.
   - *Question:* How do you use the concept of the "Cone of Uncertainty" to explain why a single absolute number is impossible at this stage, and how can you propose a phased feasibility contract instead?